Freenet User Manual

Everything you need to install, use, check, fix, and grow with your Freenet node.

Manual revision	**1.1**
Written against Freenet	**v0.2.133**
Date	**2026-09-05**
Source	`docs/user-manual/` in freenet-core

Contents

About This Manual

This manual is a **living document**. It lives in the Freenet source repository (`docs/user-manual/freenet-user-manual.md`) and is updated alongside the software itself, so what you read here always describes a specific, named Freenet release (see the cover page). A print-ready PDF is regenerated from this file with the included `build-pdf.py` script.

How updates are highlighted

Every time the manual is revised, changes are marked so returning readers can see at a glance what is new — and so the manual doubles as a **visual growth chart** of both the software and your own journey with it:

- NEW marks a section added in the current revision.
- UPDATED marks a section whose content changed in the current revision.
- **Appendix E** contains the full revision history and the growth chart: which sections existed at each revision, and how coverage has expanded over time.

This is revision 1.1 — the first revision to exercise this machinery. Badges in this edition mark what changed between Freenet v0.2.123 and v0.2.133 (plus a small number of purely editorial additions, identified as such in Appendix E's delta ledger).

How this manual is organized — the reader's growth path

The manual is arranged as a ladder. Start at the level that matches you today and climb as you advance:

Level	You want to…	Read
🟢 **Newcomer**	Understand Freenet and get it running	Parts I–II
🔵 **Everyday user**	Use apps, keep the node healthy	Parts II–IV
🟠 **Operator**	Configure, diagnose, fix, and upgrade	Parts III–VI
🔴 **Power user / developer**	Tune resources, manage secrets, build apps	Part VII

Conventions

- Commands you type are shown `like this` . Lines starting with `$` are typed at a terminal (don't type the `$`).
- `freenet` is the node binary; `fdev` is the separate developer tool.
- File paths are given for Linux first; Appendix B lists the macOS and Windows equivalents.
- Freenet is under active development. Where behavior is version-dependent, the manual says so and names the release.

Part I — Understanding Freenet

1. What Freenet Is

Freenet is a peer-to-peer network that turns the computers of its users into a single, resilient, distributed platform on which anyone can build and use decentralized services. There are no central servers to fail, censor, or de-platform you: every peer contributes storage, bandwidth, and computation to a fault-tolerant collective, and every application built on Freenet is interoperable by default.

From your point of view as a user, Freenet is deliberately boring:

1. You install a small program (the **Freenet node**, the `freenet` binary).
2. It runs quietly in the background as a system service.
3. You open Freenet applications **in your normal web browser**, served by your own node at a local address — no special browser, no plugins.

Under the hood, your node connects over UDP to other peers, forming a "small-world" network that can find and synchronize data quickly without any central index.

1.1 The pieces you'll encounter

Piece	What it is
Node (peer)	The `freenet` program running on your machine. Stores data, routes traffic, executes contracts.
Gateway	A publicly reachable node that helps new peers join the network. Your node fetches a list of gateways from `freenet.org` on first start.
Contract	A small WebAssembly program plus its associated public state, replicated across peers. Contracts define what state is valid and how it merges — this is how Freenet keeps data consistent without servers.
Delegate	A WebAssembly program that runs **only on your node**, managing your private data and keys — never replicated to other peers.
App / web app	A normal web application (HTML/JS/WASM) published on Freenet and loaded through your local node in your browser.
Dashboard	Your node's local web page at `http://127.0.0.1:7509/` showing status and available apps.
WebSocket API	The local API (port 7509) that apps and tools use to talk to your node.

1.2 What Freenet is not

- It is **not** a VPN or an anonymizing proxy for the ordinary web.
- It is **not** the same software as the 1999 "Freenet" (that project is now called Hyphanet); this is a ground-up redesign, sometimes called "Freenet 2023" or Locutus during development.
- It does **not** require you to be a developer. Running a node and using apps needs no programming at all.

1.3 Where to learn more

- Website: https://freenet.org/
- Whitepaper (architecture deep-dive): https://freenet.org/whitepaper/
- Community chat (Matrix): `#freenet:matrix.org`
- Source code: https://github.com/freenet/freenet-core

Part II — Getting Started

2. Installing Freenet

2.1 System requirements

- **OS:** Linux (x86_64/aarch64), macOS, or Windows.
- **Memory:** 1 GB+ available RAM recommended.
- **Disk:** by default Freenet will use up to about 1 GiB for hosted contract state (configurable — see §17), plus logs and caches.
- **Network:** an ordinary internet connection. Freenet uses UDP (default port 31337) and works behind most home routers/NAT without configuration.

2.2 Recommended install (Linux and macOS)

Run the official installer:

```
curl -fsSL https://freenet.org/install.sh | sh
```

The installer:

- downloads the latest signed release binary into `~/.local/bin/`;
- sets up a **supervised service** (systemd on Linux, launchd on macOS) so the node starts automatically and — crucially — **auto-updates** (see Part VI);
- on Linux, prefers a system-wide service when it can elevate (root or sudo), otherwise installs a user service with *lingering* enabled so the node keeps running when you're logged out.

You can customize it with environment variables before the command:

Variable	Effect
`FREENET_INSTALL_DIR`	Install directory (default `~/.local/bin`)
`FREENET_VERSION`	Install a specific version instead of the latest
`FREENET_NO_SERVICE=1`	Install the binary only, **no** service. The node will not auto-update until you set up supervision.

> **Why the service matters:** a Freenet node never replaces its own binary while running. When it detects a new release it *exits with code 42* and relies on its supervisor to run `freenet update` and restart it. Without a supervisor, a node detects the update, exits… and stays stopped on the old version. Install the service unless you know you want manual control.

2.3 Desktop apps: macOS and Windows UPDATED

macOS now has a first-class app: download the DMG from https://freenet.org/, drag Freenet to Applications, and launch it. The app runs the node under a menu-bar wrapper (the same supervision — auto-update, crash backoff, log capture — that systemd provides on Linux) and, on first launch, automatically puts the `freenet` and `fdev` command-line tools on your `PATH` (symlinked into `/usr/local/bin`), so the terminal commands in this manual work without any shell configuration. *(App since v0.2.125; automatic CLI setup since v0.2.133.)* The §2.2 installer script also still works on macOS if you prefer a service without the app.

Windows: download the Windows installer from https://freenet.org/. The node runs under a **tray application**: a Freenet icon in the system tray with Start/Stop/Dashboard controls, with the same supervision as above. On first launch it opens the dashboard in your browser. Since v0.2.131, Windows release binaries are **Authenticode-signed**, so SmartScreen warnings about an unknown publisher no longer apply to current releases.

2.4 Installing from source (advanced)

With a Rust toolchain installed:

```
git clone https://github.com/freenet/freenet-core.git
cd freenet-core
cargo install --path crates/core     # the node:  freenet
cargo install --path crates/fdev     # dev tool:  fdev (optional)
```

Then set up the service yourself: `freenet service install` (add `--system` for a root-owned, system-wide service). Note that **locally built ("dirty") dev builds disable auto-update** on purpose, so an update can never clobber your local changes.

2.5 Verifying the install

```
$ freenet --version
```

This prints the version plus the exact git commit and build timestamp — you'll use it again in the self-check (Part IV).

2.6 Docker NEW

Since v0.2.133 there is an official container image, `ghcr.io/freenet/freenet-core`, published for every stable release (tags: exact version like `v0.2.133`, minor series `0.2`, and `latest`; built for `linux/amd64` and `linux/arm64`):

```
docker run -d --name freenet-node --network host \
    -v freenet-data:/data --restart unless-stopped \
    ghcr.io/freenet/freenet-core:latest
```

Then open `http://127.0.0.1:7509/` as usual. Three things to know:

- **The container self-updates.** The image's entrypoint plays the supervisor role (§2.2): it catches the node's exit-42 "update me" signal, applies the update, and restarts. You do not need Watchtower, cron, or manual `docker pull` to stay current — a container started once keeps itself up to date.
- The node runs from `/data/bin/freenet` on the **volume**, not from the image layer, so applied updates survive `docker compose down && up`. On start, the newer of (image binary, volume binary) wins — pulling a newer image never rolls a self-updated node backwards.
- `docker exec freenet-node freenet --version` reports the version actually running, which may be ahead of what the image shipped with.

A ready-made `docker-compose.yml` lives in `docker/freenet-node/` in the source repository, alongside the full container documentation.

3. First Run and the Dashboard UPDATED

If you installed with the service (the default), the node is already running. Open the **dashboard**:

> **http://127.0.0.1:7509/**

The dashboard is served by your own node, on your own machine — it works even though it's "a website" because your node includes a small local web server. From here you can see node status and launch Freenet applications. The dashboard has grown considerably since v0.2.123: it now shows a **measured GET success rate** (real fetch outcomes, not a synthetic health verdict), offers a **per-contract detail page** at `/contract/{key}` for anything your node hosts, follows your OS light/dark theme, and its tables filter and collapse for small screens.

If you skipped the service, you can run a node in the foreground:

```
$ freenet network
```

Leave it running in that terminal and open the dashboard as above. (`freenet` with no subcommand does the same thing — network mode is the default.)

The first startup fetches the current **gateway list** from `https://freenet.org/keys/gateways.toml`, connects to a gateway over UDP, and joins the network. Within a short time the node acquires peer connections of its own and no longer depends on the gateway.

4. Using Freenet Applications

Freenet apps are ordinary web apps loaded through your node. The flagship application today is **River** (https://freenet.org/) — decentralized group chat with invitation-based rooms, no server, no account with any company.

General pattern for any Freenet app:

1. Make sure your node is running (dashboard loads).
2. Open the app's local URL in your browser (linked from the dashboard or from the app's own site).

3. The app talks to your node over the local WebSocket API; your node fetches and synchronizes the app's contracts with the network.

Because apps run against *your* node, they keep working as long as your node does — there is no "server down" failure mode, only your own machine.

Part III — Operating Your Node

5. The Service: Day-to-Day Control

All service management goes through `freenet service …`. On Linux these commands drive systemd (user service by default; add `--system` everywhere if you installed system-wide); on macOS, launchd; on Windows, the tray wrapper.

Command	What it does
`freenet service status`	Health summary: is the service installed, running, which binary/version, key paths. **Start here whenever something feels off.**
`freenet service start` / `stop` / `restart`	Start, stop, or bounce the node.
`freenet service logs`	Follow the node's log output live.
`freenet service logs --err`	Only error-level logs.
`freenet service disable`	Keep the node stopped **persistently** — it will not come back on reboot or re-login until you re-enable. (The supervisor stays installed; only the node idles.)
`freenet service enable`	Undo `disable` and start the node again immediately.
`freenet service doctor`	Repair a wedged install (see §11.3).
`freenet service report`	Generate/upload a diagnostic report (see §10).
`freenet service install` / `uninstall`	Add or remove the service itself.

Stopping vs disabling: `stop` is temporary — the service starts again on the next boot. `disable` writes a persistent marker that makes the node refuse to run until you `enable` it again. Use `disable` when you want Freenet off for a while on a laptop, for example.

6. Configuration

6.1 Where things live (Linux)

What	Default location
Config directory	`~/.config/freenet/`
Data directory (contract state, databases, caches)	`~/.local/share/freenet/`
Logs	under the data/log dir shown by `freenet service status`
Gateway cache	`gateways.toml` in the config directory
Secrets (encrypted delegate keys)	`secrets/` under the data directory

Override with `--config-dir`, `--data-dir`, `--log-dir` flags or the `CONFIG_DIR`, `DATA_DIR`, `LOG_DIR` environment variables. See Appendix B for macOS/Windows paths.

6.2 Options you're most likely to touch

Every option can be given as a CLI flag or an environment variable:

Option	Default	Meaning
`--ws-api-port`	`7509`	Local WebSocket API + dashboard port.
`--network-port`	`31337`	UDP port for peer-to-peer traffic.
`--address`	`::` (dual-stack)	Bind address for the network listener.
`--log-level` (`LOG_LEVEL`)	`info`	`error`, `warn`, `info`, `debug`, `trace`.
`--max-hosting-storage`	1 GiB	Budget for hosted contract state; least-valuable contracts are evicted beyond this (§17).
`--max-blocking-threads`	2×CPU (4–32)	Worker threads for WASM execution.

6.3 Ports and firewalls

- **7509/TCP, localhost only** — dashboard and WebSocket API. Do **not** expose this to the internet; it is meant for your own browser and local tools.
- **31337/UDP** — peer traffic. Freenet's transport performs NAT traversal, so most home users need no router configuration. Opening/forwarding 31337/UDP can improve connectivity but is not required unless you run a gateway.

6.4 Running a gateway (advanced)

A gateway is a node with a stable public address that helps new peers join. To run one you need a public IP and an open UDP port, and you start the node with:

```
freenet network --is-gateway \
    --public-address <your.public.ip> --public-port 31337
```

Gateways run "isolated" — they do not themselves bootstrap through other gateways. Operating a public gateway is a service to the network; if you're interested, coordinate with the project through Matrix so your gateway can be listed in the official index.

Part IV — The Operational Self-Check

Run this whenever you want to confirm your node is healthy — after installing, after an upgrade, after a reboot, or just periodically. The whole check takes about two minutes. Each step says what **good** looks like and where to go if the step fails.

7. The Ten-Step Health Check UPDATED

Step 1 — What am I running?

```
$ freenet --version
```

✅ Prints a version (e.g. `0.2.123`) with a git commit and build timestamp. ❌ "Command not found" → the install directory (default `~/.local/bin`) isn't on your `PATH`, or Freenet isn't installed. → §2.

Step 2 — Is the service healthy?

```
$ freenet service status        # add --system for a system-wide install
```

✅ Service installed, active/running, recent start time, paths listed. ❌ Not installed → §2.2 / `freenet service install`. ❌ Installed but stopped → `freenet service start`, then re-check. If it stops again, go to §11.1.

Step 3 — Does the node answer locally?

Open `http://127.0.0.1:7509/` in a browser, or:

```
$ curl -sf http://127.0.0.1:7509/ >/dev/null && echo OK
```

✅ Dashboard loads / `OK`. ❌ Connection refused → the node isn't running or the WS port was changed → §11.2.

Step 4 — Is it the version I think it is?

Compare `freenet --version` (the binary on disk) with the running service's version shown in `freenet service status`. ❌ They differ → the node is running an old binary; restart the service, and if it persists run `freenet service doctor` (§11.3).

Step 5 — Are there peers?

```
$ fdev query
```

✅ Shows open connections to other peers. ❌ No connections after several minutes → §11.4 (connectivity). *(No `fdev`? It's optional — the dashboard also reflects network state, and you can skip to Step 6.)*

Step 6 — Anything alarming in the logs?

```
$ freenet service logs --err
```

✅ Quiet, or only transient warnings. ❌ Repeated errors — note the message and match it against §11's symptom table.

Step 7 — Am I up to date?

```
$ freenet update --check
```

✅ "Already up to date" (or it names a newer version — see Part VI to decide). Supervised installs update themselves; if this reports you are several versions behind *and* you installed the service, something is blocking updates → §11.3.

> ⚠️ **This step matters more than it used to** (v0.2.133): the network now enforces a minimum compatible version at the connection handshake. A node that falls too far behind isn't just missing features — every peer refuses its connections and it is cut off entirely. See §13.

Step 8 — Disk headroom.

```
$ df -h ~/.local/share/freenet
```

✅ Free space comfortably above 1 GiB. Freenet bounds its own hosting storage (§17), but logs and databases still need room.

Step 9 — Clean restart survives. *(optional but recommended after upgrades)*

```
$ freenet service restart && sleep 10 && freenet service status
```

✅ Comes back to running; dashboard loads again.

Step 10 — Deep diagnostics. *(optional)*

```
$ fdev diagnostics
```

✅ Prints detailed node state: network info, subscriptions, metrics, uptime. This is the same information a diagnostic report captures (§10).

> **Make it a habit:** steps 1–3 alone (version, service status, dashboard) catch the vast majority of problems and take under 30 seconds.

8. Reading the Signals

Three numbers the node uses to talk to its supervisor are worth knowing, because you will see them in logs and `systemctl` output:

Exit code	Meaning
`0`	Clean, requested shutdown. Normal.
`42`	"A new release is available — update me." The supervisor runs `freenet update` and restarts. Seeing this in logs during an upgrade is **normal and healthy**.
`43`	"Another Freenet node is already running" (port conflict). The wrapper resolves genuine orphan conflicts itself; if you see this repeatedly, see §11.2.

9. Correct what needs correcting — the principle

The self-check is designed so that **every failing step points at a fix**:

- Steps 1–2 fail → installation problem → Part II.
- Steps 3–4 fail → service/binary problem → §11.1–11.3, `freenet service doctor`.
- Step 5 fails → network problem → §11.4.
- Step 7 fails → update pipeline problem → Part VI.
- Anything you can't classify → generate a diagnostic report (§10) and ask for help (§12).

10. Diagnostic Reports

When you need help — or want a snapshot of node health for your own records — generate a diagnostic report:

```
$ freenet service report
```

The report bundles: system info (OS, architecture, hostname), exact version and build info, recent logs (main and error), your config file, and — if the node is running — its live network status queried over the local WebSocket API. You'll be prompted for a one-line description of the problem, then the report is uploaded to the Freenet diagnostic server and you get a reference to share in chat or an issue.

Useful variants:

```
$ freenet service report --local report.json   # save locally, upload nothing
$ freenet service report -m "node has no peers since yesterday"
$ freenet service report --no-message          # skip the description prompt
```

Privacy note: the report contains your logs and config. Use `--local` first if you want to inspect exactly what would be sent. If the node isn't reachable when the report is generated, the report says *why* (connection refused, timeout, …) rather than silently omitting network status — that distinction itself is a useful diagnostic.

Part V — Correcting Problems

11. Troubleshooting by Symptom

11.1 The node won't start, or keeps stopping

1. `freenet service logs --err` — read the last few lines; the node states its reason for exiting.
2. Check for the **disable marker**: if you (or someone) ran `freenet service disable`, the node refuses to run by design. Fix: `freenet service enable`.
3. Crash loops are handled with backoff by the supervisor (10 s doubling up to 5 min). If crashes started **right after an update**, do nothing for a few minutes: the node counts probation crashes and **rolls itself back automatically** to the previous known-good version (§14.2). If crashes are unrelated to an update, capture a report (§10) and ask for help (§12).

11.2 Port already in use / exit code 43 / dashboard unreachable

- Another process (often an orphaned old Freenet process) holds the network port. The supervisor kills genuine orphans automatically (up to 3 attempts); if the conflict persists, a *different* application owns the port — find it (`ss -ulpn | grep 31337` on Linux) and either stop it or move Freenet with `--network-port`.
- Dashboard specifically unreachable while the node runs: confirm the WS API port (default 7509) wasn't customized, and that you're browsing from the same machine — the API binds to localhost.

11.3 Node seems stuck on an old version

Symptoms: `freenet update --check` says a newer version exists for days; or `freenet --version` (binary) disagrees with the running service.

```
$ freenet service doctor      # add --system for system-wide installs
```

`doctor` re-points the service at the current binary, reaps stale orphaned `freenet network` processes still holding the port on an old binary, and restarts cleanly. It fixes the classic "frozen on an old version" state that a plain `restart` cannot. After running it, re-do self-check steps 2–4 and 7.

Also remember: **unsupervised** nodes (installed with `FREENET_NO_SERVICE=1`, or hand-run with `freenet network`) do not auto-update at all — see §14.3. And **dev/dirty builds** never auto-update by design.

11.4 No peers / can't join the network UPDATED

0. **Check your version first** (v0.2.133): the transport handshake enforces a minimum compatible version, so a node that is too far out of date is *refused by every peer* — it looks exactly like a connectivity problem but is fixed by `freenet update`. Self-check step 7 rules this out in seconds.
1. Give a fresh node a few minutes — joining requires a round-trip through a gateway.

2. Check basic connectivity: can you reach the internet at all? Does your network block outbound UDP? (Some corporate/university networks do.) Freenet requires outbound UDP; without it the node cannot join.
3. Check the log for gateway errors (`freenet service logs`). The node fetches the gateway index from `https://freenet.org/keys/gateways.toml` ; if that fetch fails it falls back to the cached `gateways.toml` in your config dir.
4. Still stuck → diagnostic report + ask in Matrix (§12).

11.5 "No file descriptors available" or resource errors

The node raises its own file-descriptor limit at startup to the kernel hard limit automatically. If you still see `EMFILE` /fd errors on an unusual setup, raise the hard limit for the service (systemd: `LimitNOFILE=`), then restart.

11.6 Disk usage growing UPDATED

Hosted contract state is bounded (default 1 GiB; overall disk budget is additionally capped — §17) and evicted least-valuable-first. Since v0.2.130, **logs are bounded too**: the node prunes its own log directory to a 512 MiB budget in the background, overridable with the `FREENET_LOG_DIR_MAX_BYTES` environment variable (shrink it on a quiet peer to hand back disk, or widen it while chasing an intermittent fault). The compiled-WASM cache is also now bounded by disk headroom, not just RAM (v0.2.126). If disk still grows unboundedly, check the data directory with `du` and file an issue — nothing is supposed to grow without a budget anymore.

12. Getting Help

1. **Generate a diagnostic report first** (§10) — it answers 90% of the questions a helper would ask.
2. **Matrix chat:** `#freenet:matrix.org` — the developers are active here.
3. **GitHub issues:** https://github.com/freenet/freenet-core/issues — for reproducible bugs. Include your report reference, `freenet --version` output, OS, and what the self-check showed.
4. **freenet.org** — links to current docs, FAQ, and community channels.

Part VI — Upgrading

13. How Auto-Update Works UPDATED

> **Staying current is no longer optional** (v0.2.133). Freenet ships releases frequently — sometimes several a day — and peers are expected to converge on new releases within hours. The network enforces a `min-compatible-version` floor **as a hard gate at the transport handshake**: once a node drops below the floor, every peer refuses its connections and it is cut off. The supervised auto-update pipeline below is what keeps that from ever happening to you.

Freenet releases frequently, and the update system is designed so a supervised node **keeps itself current with zero attention from you** — while protecting you from a bad release. The pipeline:

1. **Detection.** The running node notices a new official release and exits with code **42** ("update me"). It never overwrites itself while running.
2. **Download & verify.** The supervisor catches exit 42 and runs `freenet update`, which downloads the release, checks it against the release's `SHA256SUMS.txt` manifest, and **verifies the manifest's ed25519 signature against a public key baked into your binary**. An artifact that fails verification is never installed.
3. **Swap & restart.** The verified binary replaces the old one (which is kept as the known-good fallback) and the service restarts on the new version.
4. **Probation.** For a short period after an update the node is "on probation": crashes are counted.
5. **Automatic rollback.** If the new version crash-loops during probation, the updater **restores the previous known-good binary**, pins the bad version as known-bad on this node (so it won't be retried), and restarts. This works even with no network connection.

You can watch all of this happen in `freenet service logs` — a healthy update shows an exit-42, an update run, and a restart on the new version.

14. Updating Manually

14.1 Commands

```
$ freenet update --check    # just tell me; change nothing
$ freenet update            # download, verify, install the latest release
$ freenet update --force    # reinstall even if already on the latest
$ freenet update --quiet    # no interactive output (for scripts)
```

After a manual update, restart the service (`freenet service restart`) and run self-check steps 1–4.

14.2 After a bad update

Normally you do nothing: automatic rollback (§13 step 5) handles a crash-looping release, and the node then simply skips that version. When a fixed release ships, update as usual — `freenet update --force` if the fixed release reuses a version number the node has pinned as known-bad.

14.3 Keeping an *unsupervised* node current

A node you run by hand (`freenet network` in a terminal, or installed with `FREENET_NO_SERVICE=1`) will detect an update, print that it needs one, and exit — and then nothing restarts it. Your choices:

- run `freenet update` yourself when prompted in the logs, then start the node again; or
- (better) convert to a supervised install once: `freenet service install`.

15. Upgrading *Yourself*: as you advance

The manual's growth path (About This Manual) mirrors a real progression:

1. **Newcomer → Everyday user:** make the self-check (Part IV, steps 1–3) a monthly habit; learn to read `freenet service logs`.
2. **Everyday user → Operator:** learn `service doctor`, diagnostic reports, and the update pipeline; skim your config (§6) so defaults are choices, not accidents.
3. **Operator → Power user:** take control of secrets (§16) and resource budgets (§17); consider running a gateway (§6.4).
4. **Power user → Developer:** install `fdev` and work through the developer guide (§18) and the freenet-ping example app.

Each new manual revision highlights what's changed (badges + Appendix E), so re-skimming the manual after an upgrade shows you exactly which capabilities are new since you last read it — your growth chart and the software's, on the same page.

Part VII — Advancing: Power Use

16. Secrets: Protecting Your Keys and Private Data UPDATED

Delegates keep your private data (identity keys, chat-room keys, credentials) **encrypted at rest** on your node. The design in one paragraph: every secret is encrypted with a per-delegate key (DEK), and every DEK is derived from a single master **Key Encryption Key (KEK)** held in a pluggable backend. Manage it with the `freenet secrets` commands:

Command	Purpose	Node state
`freenet secrets status`	Show the active KEK backend and the KEK fingerprint (never the key itself).	any
`freenet secrets provision`	Choose a KEK backend **before first start** — e.g. opt into the OS keyring.	before first run
`freenet secrets rotate`	Generate a new KEK and re-encrypt every secret and snapshot under it.	**stopped**
`freenet secrets migrate`	Move the KEK between backends (`file` ↔ `keyring` ↔ `systemd`).	**stopped**
`freenet secrets snapshots`	List per-secret backup history (metadata only — never plaintext).	any
`freenet secrets restore`	Roll a secret back to an earlier snapshot (the current value is snapshotted first, so this is reversible).	**stopped**
`freenet secrets export`	Pack a scope's secrets into one **encrypted, portable bundle** — e.g. to move your identity to a new machine.	**stopped**
`freenet secrets import`	Import a bundle produced by `export` on another node.	**stopped**

Backends: `keyring` (OS keychain / Credential Manager — strongest for desktops; the auto-resolver deliberately does *not* pick it silently, because it can trigger an OS consent prompt: opt in with `provision`), `systemd` (systemd-credentials — good for Linux servers), `file` (portable default).

Moving to a new machine: stop the node → `secrets export` → copy the bundle → install Freenet on the new machine → `secrets import` → start. The bundle is always encrypted (passphrase-derived key), so it is safe to carry on a USB stick — but treat it like the keys it contains.

Make backups a habit, not just a migration step. Your delegate keys are the one thing on your node that cannot be re-fetched from the network — if the disk dies and there is no bundle, identities and room keys are gone permanently. Run `secrets export` after creating any identity you care about and after any significant new secret, and store the bundle (plus its passphrase, separately) somewhere that doesn't share fate with the machine. Everything else — hosted contract state, caches, the binary — is replaceable; the secrets are not.

Full operator documentation: `docs/secrets-at-rest.md` in the source repository.

17. Storage and Resource Tuning UPDATED

Your node hosts a share of the network's contract state. Three dials bound it:

Dial	Default	Notes
`--max-hosting-storage`	1 GiB	RAM-style budget on tracked contract state; beyond it, least-valuable contracts are evicted and their disk reclaimed.
`--hosting-disk-pct`	0.5	Fraction of the disk capacity available to Freenet used to size the aggregate disk budget. The effective budget is the **minimum** of the two budgets.
`--max-hosting-disk`	32 GiB	Hard cap on the disk budget regardless of disk size.

Other useful dials:

- `--module-cache-budget-bytes` — cache of compiled contract WASM. Default scales with RAM (RAM/8, clamped 64 MiB–4 GiB). Raise it on a busy node with many contracts to avoid recompilation churn.
- `--max-blocking-threads` — WASM execution parallelism (default 2×CPU cores, clamped 4–32).

Since v0.2.126–0.2.127 the memory side is smarter than a fixed ceiling: the node bounds overall peer memory (~2 GiB cap), sizes hosting eviction pressure from **live memory measurements** rather than a hardcoded per-contract overhead estimate, and bounds the compiled-WASM cache by actual disk headroom as well as RAM. In practice this means a node on a small VPS behaves itself without hand-tuning, and eviction responds to real pressure instead of worst-case guesses.

The defaults are deliberately conservative: a stock node donates a bounded, predictable amount of your disk and memory. Raising the budgets makes your node a more valuable network citizen; it never grows unbounded either way.

18. First Steps as a Developer UPDATED

Everything on Freenet — every app, every chat room — is contracts plus delegates plus a web front-end, and the tooling is a single CLI:

```
cargo install --path crates/fdev     # or: cargo install fdev
```

`fdev` command	Purpose
`fdev new`	Scaffold a new contract or web-app package.
`fdev build`	Build a contract/delegate/web-app to WASM.
`fdev publish`	Publish a contract or app to the network (or your local node).
`fdev query`	Show your node's open peer connections.
`fdev diagnostics`	Detailed node state: network, subscriptions, metrics.
`fdev inspect`	Compute a contract's ID without publishing.
`fdev verify-merge`	Check that a contract's merge obeys the laws the network requires (order-independence, associativity, idempotence) — the *same* verifier the network runs. A contract that fails here cannot converge on the network. Formerly named `conformance`. *(New in v0.2.129–0.2.133.)*
`fdev website init/publish/update`	Keypair-based publishing of static websites on Freenet.
`fdev commands get/subscribe/update`	Raw contract operations against a node's WebSocket API.

The recommended path: run a node in **local mode** (`freenet local` — a sandboxed node with no network traffic), then build and publish against it, then graduate to the real network. Study the example app `apps/freenet-ping` in the source tree — a minimal but complete contract + client that exercises the full put/subscribe/update cycle. The developer-facing architecture documentation lives in `docs/architecture/` and the whitepaper.

Part VIII — Uninstalling

Freenet removes cleanly:

```
$ freenet uninstall            # interactive: asks what to do with your data
$ freenet uninstall --keep-data   # remove service + binaries, keep data/config
$ freenet uninstall --purge       # remove everything, including data & logs
```

Add `--system` if you installed the system-wide service. To remove only the service but keep the binary (e.g. switching to hand-run mode), use `freenet service uninstall` instead.

> ⚠️ `--purge` deletes your **secrets** too — any identities or room keys held by delegates are gone permanently unless you exported a bundle first (§16).

Appendices

Appendix A — CLI Quick Reference

Command	One-liner
`freenet` / `freenet network`	Run the node (network mode).
`freenet local`	Run a sandboxed local-only node.
`freenet --version`	Version + git commit + build timestamp.
`freenet service install [--system] [--no-linger]`	Install as a supervised service.
`freenet service status/start/stop/restart`	Control the service.
`freenet service enable/disable`	Persistently allow/forbid the node to run.
`freenet service logs [--err]`	Follow logs.
`freenet service doctor [--system]`	Repair a wedged install.
`freenet service report [--local PATH] [-m MSG]`	Diagnostic report.
`freenet update [--check] [--force] [--quiet]`	Update the binary.
`freenet secrets status/provision/rotate/migrate/snapshots/restore/export/import`	Key & secret management (§16).
`freenet uninstall [--purge | --keep-data] [--system]`	Remove Freenet.
`fdev …`	Developer tool (§18).

Appendix B — Default Ports and File Locations UPDATED

Ports

Port	Protocol	Scope	Purpose
7509	TCP (HTTP/WebSocket)	localhost only	Dashboard + client API
31337	UDP	internet	Peer-to-peer transport

Directories (defaults; every one is overridable — §6.1)

OS	Config	Data
Linux	`~/.config/freenet/`	`~/.local/share/freenet/`
macOS	`~/Library/Application Support/The-Freenet-Project-Inc.Freenet/`	same tree
Windows	`%APPDATA%\The Freenet Project Inc\Freenet\config\`	`%APPDATA%\The Freenet Project Inc\Freenet\data\`

Binary (installer default): `~/.local/bin/freenet` on Linux/macOS. The macOS app symlinks the CLI tools into `/usr/local/bin`. Docker keeps everything on the `/data` volume (binary at `/data/bin/freenet`).

Appendix C — Exit Codes

Code	Meaning	Action
0	Clean shutdown	none
42	Update requested	supervisor runs `freenet update`; normal during upgrades
43	Another node already running	wrapper auto-resolves orphans; else §11.2
other	Crash	supervisor restarts with backoff; probation crashes may trigger auto-rollback (§13)

Appendix D — Glossary

- **AOF** — append-only file; the node's local event log format.
- **Contract** — WASM program + replicated public state; the unit of shared data on Freenet.
- **Dashboard** — the node's local status/app page at `http://127.0.0.1:7509/`.
- **DEK / KEK** — data-encryption key (per delegate) / key-encryption key (master); see §16.
- **Delegate** — WASM program managing private data, runs only on your node.
- **Gateway** — publicly reachable node that bootstraps new peers.
- **Local mode** — sandboxed single-node mode for development (`freenet local`).
- **Node / peer** — a running `freenet` instance participating in the network.
- **Probation** — the crash-watch window right after an update; see §13.
- **Supervisor** — systemd / launchd / tray wrapper that restarts and updates the node.
- **WS API** — the node's local WebSocket client API (port 7509).

Appendix E — Manual Revision History & Growth Chart

The manual grows with the software and with its readers. Each row records a revision; the chart shows how coverage has expanded. New and updated sections in the *current* revision are additionally badged inline throughout the text.

Manual rev	Date	Freenet version	What changed
1.0	2026-08-25	0.2.123	Initial full manual: concepts, install, operations, ten-step self-check, troubleshooting, auto-update & rollback, secrets, tuning, developer intro, appendices.
1.1	2026-09-05	0.2.133	First living revision: Docker install, macOS app, version-floor warning, bounded logs, memory-aware budgets, dashboard growth, `fdev verify-merge`, backup guidance. Full delta ledger below.

Revision 1.1 delta ledger (every badge in this edition traces to a row here; "driver" names the upstream release or marks the change as editorial):

Section	Badge	Change	Driver
§2.3 Desktop apps	UPDATED	macOS DMG app with automatic CLI `PATH` setup; Windows binaries Authenticode-signed	v0.2.125/0.2.133; v0.2.131
§2.6 Docker	NEW	Official self-updating container image `ghcr.io/freenet/freenet-core`	v0.2.133
§3 Dashboard	UPDATED	Measured GET success rate, `/contract/{key}` detail pages, OS theme, filterable tables	v0.2.129–0.2.130
§7 Health check	UPDATED	Step 7 warning: compatibility floor makes staying current mandatory	v0.2.133
§11.4 No peers	UPDATED	New first cause: node below the network's minimum compatible version	v0.2.133
§11.6 Disk usage	UPDATED	Logs auto-pruned to 512 MiB, `FREENET_LOG_DIR_MAX_BYTES` override; WASM cache disk-bounded	v0.2.130; v0.2.126
§13 Auto-update	UPDATED	`min-compatible-version` enforced as a hard gate at the transport handshake	v0.2.133
§16 Secrets	UPDATED	Backup-as-a-habit guidance (secrets are the only unrecoverable data)	editorial
§17 Tuning	UPDATED	Live-memory-aware hosting budgets, ~2 GiB peer memory cap, disk-headroom-bounded WASM cache	v0.2.126–0.2.127
§18 Developer	UPDATED	`fdev verify-merge` (merge-law verifier, formerly `conformance`)	v0.2.129–0.2.133
Appendix B	UPDATED	macOS `/usr/local/bin` symlinks; Docker `/data` volume paths	v0.2.133

Coverage growth chart (sections present per revision):

Reading the chart: each future revision adds a row; the bar length is proportional to the number of sections, so the chart literally shows the manual — and the platform — growing. Compare any two revisions via the badges listed in their rows to see exactly what to re-read.
